

Analysis and QUIC-Based Interruption Recovery in P2P Conferencing under WiFi-to-Cellular Transitions

Anonymous Author(s)

Abstract

Major conferencing services such as Zoom and Microsoft Teams use peer-to-peer (P2P) connection for one-to-one calls under certain conditions to reduce server-side media processing costs. However, P2P connectivity is fragile under an endpoint's WiFi-to-cellular transitions, where IP address changes often cause noticeable service interruptions. In this paper, we conduct a packet-level analysis of recovery procedures in existing P2P conferencing systems under IP address changes and find that the delay before the remote peer becomes aware of the address change dominates the interruption time. Motivated by these findings, we propose QSwitch, a seamless recovery approach under WiFi-to-cellular transitions for P2P conferencing. QSwitch leverages QUIC for both signaling and media delivery, taking advantage of its connection migration capability to enable an immediate fallback to a server-relayed path. To avoid confounding factors in comparisons with commercial applications, we implement a WebRTC-based baseline that performs relay fallback under the same handover trigger. Our evaluation shows that QSwitch reduces the median interruption time by 58.1% compared to the baseline.

1 Introduction

Real-time online conferencing has become essential for personal and professional communication [8, 18]. To reduce the growing infrastructure costs of large-scale media processing [4], many systems such as Zoom [1], Microsoft Teams [3], Jitsi Meet [9] and Whereby [5] employ P2P connection for one-to-one calls under certain conditions¹².

However, P2P connectivity becomes fragile when a peer changes its network from WiFi to cellular. When a peer changes networks, its IP address changes and the existing NAT mapping becomes invalid. Restoring connectivity requires creating new mappings for the new address and exchanging updated addresses through a signaling channel, effectively repeating the establishment procedure. This leads to longer interruption time compared to server-centric systems, which can simply reconnect to global addresses of servers.

To cope with the interruption under network transitions, several recovery approaches are proposed, including ICE Restart [12] or ICE Renomination [17]. Although these approaches differ in their mechanisms, they share a common limitation: media delivery remains blocked until signaling exchanges and transport-layer procedures complete.

To better understand the primary causes of the interruption, we analyze the recovery procedures and their overhead under IP address changes in P2P calls in Zoom and Microsoft Teams. We first investigate how each system recovers from an interruption by examining packet traces during a P2P call. We then decompose the recovery procedure into three phases: (i) the delay from the IP address change to recovery initiation, (ii) the delay until the remote peer becomes aware of the address change, and (iii) the delay until media delivery resumes. By measuring these delays, we find that the second phase dominates the overall interruption time. In Zoom, this delay stems from the sender continuing to use the stale P2P path, while in Microsoft Teams it stems from repeated TCP [7] and TURN [16] procedures before the peer is notified.

Motivated by these findings, we design QSwitch, a recovery approach for P2P conferencing under WiFi-to-cellular transitions. QSwitch leverages QUIC [10] to allow media delivery to continue across IP address changes without blocking on P2P re-establishment. When an IP address change occurs, QSwitch immediately restores connectivity via the server by utilizing QUIC's connection migration feature. Media transmission is then promptly migrated to the relay path, while QSwitch concurrently re-establishes a P2P path. Once the P2P path is restored, media traffic is switched back.

We implement an end-to-end prototype of QSwitch and evaluate it under WiFi-to-cellular transitions in one-to-one calls. To provide a controlled comparison, we also implement a WebRTC baseline that performs relay fallback using ICE renomination under the same handover trigger as the QSwitch prototype. This avoids confounding factors introduced by production applications. In our experiments, the median interruption time is 0.762 s for QSwitch, compared to 1.819 s for the WebRTC-based baseline.

In summary, this paper makes the following contributions:

- We conduct a packet-level analysis of existing P2P conferencing systems to characterize their behavior under IP address changes, and identify the key source of interruption.

¹Microsoft Teams enables P2P connection for one-to-one calls initiated via chat.

²Zoom enables P2P connection only when both participants are located within the same local network.

- We propose QSwitch, a recovery approach from IP address changes that leverages QUIC connection migration to rapidly fall back to server-relayed media delivery.
- We implement a WebRTC baseline and evaluate QSwitch under the same handover trigger, showing a median interruption reduction of 58.1%.

2 Background and Related Work

Many conferencing systems rely on ICE [12] to establish and maintain P2P connectivity. ICE gathers multiple candidate addresses, including local address, global address and relay address, exchanges them through a signaling server and performs connectivity checks to select a valid candidate pair for media transmission. Once a candidate pair is selected, media packets are sent over the chosen transport path.

Within ICE-based systems, two recovery approaches have been proposed. **ICE Restart** is the standard recovery approach defined in the ICE specification [12]. It regenerates ICE credentials and performs a new offer/answer exchange, effectively repeating the connectivity establishment process. Because it requires new signaling, candidate gathering, and connectivity verification, media delivery remains blocked until the entire procedure completes. **ICE Renomination** [17] reduces the overhead existed in ICE Restart by reusing previously gathered candidate pairs without performing a full restart. However, in practical implementations, recovery may still be delayed by signaling reconnection, TURN re-authentication, and connectivity checks before media transmission resumes.

Beyond conferencing systems, **server-relayed fallback** approach is used by several services including Tailscale [19]. In this approach, endpoints abandon P2P path upon address change and start packet transmission via an intermediate server. Since this approach does not seek to establish P2P path after an IP address change, the recovery procedure can be simple and rapid.

As far as our knowledge, there is no prior work examining how deployed P2P conferencing systems actually recover from IP address changes at the packet level. In particular, the sources of interruption under WiFi-to-cellular transition remain insufficiently characterized. This motivates our empirical analysis of recovery procedures in existing systems at Section 3.

3 Characterizing Interruption Recovery

We first conduct a packet-level characterization of deployed P2P conferencing systems to understand how they recover from an IP address change. In this characterization experiment, an Android device switches from WiFi to cellular by

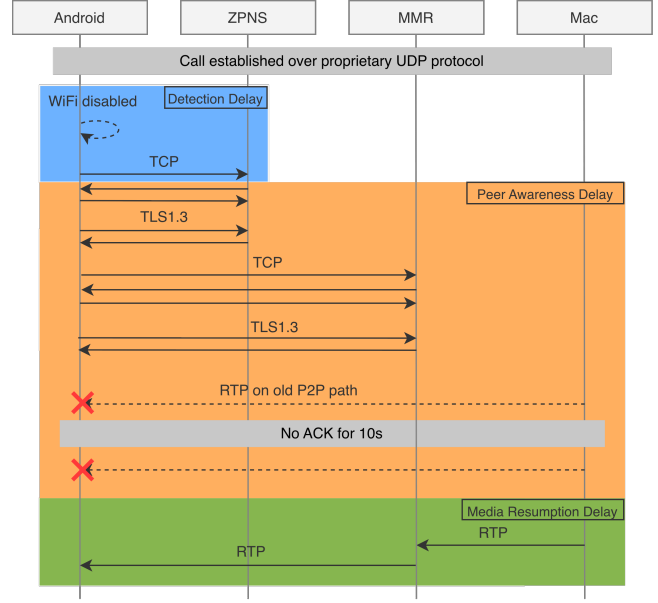


Figure 1: Zoom recovery sequence after an IP address change

disabling WiFi during an ongoing one-to-one call while a MacBook Pro sends audio.

To isolate the overhead, we decompose the recovery procedure into the following three phases:

- (1) **Detection delay:** the time from WiFi being disabled on the Android client to the initiation of the recovery procedure.
- (2) **Peer awareness delay:** the time from the initiation of the recovery procedure until the Mac client becomes aware of the Android’s IP address change.
- (3) **Media resumption delay:** the time from peer awareness until the first audio packet is received by the Android.

The timestamp of WiFi being disabled is taken from the `I/WifiService: enable=false` entry in the logcat output on the Android device. All other timing points are derived from packet traces captured with `tcpdump` by identifying protocol-specific events in the recovery procedure for each service. By correlating timestamps from packet traces with system logs, we compute the duration of each phase.

3.1 Zoom

Zoom enables P2P connection only when both participants are located within the same local network. Figure 1 shows the recovery sequence of a P2P call in Zoom under an IP address change. We observe that Zoom communicates with servers whose hostnames end with `mmr.nrt.zoom.us` (MMR) and `zpns.zoom.us` (ZPNS). The client establishes TLS 1.3 over

TCP connections to both servers, and additionally communicates with MMR over UDP. From packet traces, MMR relays RTP packets both during session establishment and after reconnection, indicating that it functions as a Multi Media Router. Because TLS 1.3 payloads are encrypted, we cannot determine which server is responsible for signaling. During Zoom calls, we do not observe any STUN or TURN traffic, suggesting that Zoom employs a proprietary protocol instead of ICE, as described in other researches [11, 13]. Media transmission is observed over both the P2P path and the MMR-relayed path even after P2P path is established. On the P2P path, each transmitted message elicits a fixed 72-byte response packet, which we identify as an acknowledgment (ACK).

After WiFi is disabled, the Android reconnects to ZPNS using TLS 1.3 over TCP. We define the SYN packet toward ZPNS as the end of the *detection delay*. The Android then reconnects to MMR. The Mac continues transmitting over the P2P path for approximately 10 s after the last ACK from the Android. Then it switches media transmission exclusively to the MMR-relayed path. We define the last packet sent over the P2P path as the end of the *peer awareness delay*. Although reconnections to ZPNS and MMR complete within 1 s after WiFi is disabled, and RTP packets flow through MMR since the beginning, the Android does not start receiving RTP packets from MMR until the Mac stops P2P transmission. The arrival of the first RTP packet via MMR at the Android marks the end of the *media resumption delay*.

As a result of the analysis, the *peer awareness delay* dominates the overall interruption time. This is because the Mac maintains the P2P path for approximately 10 s after ACK loss and the Android does not receive audio packets from the server until then.

3.2 Microsoft Teams

Microsoft Teams enables P2P connection for one-to-one calls initiated via chat. Figure 2 shows the recovery sequence of a P2P call in Microsoft Teams under an IP address change. From packet traces, we observe that a Teams client communicates with `world.relay.skype.com` (Relay Gateway) and TURN servers associated with both clients using TCP and TURN. TCP handshakes to the servers are followed by a single round trip of pseudo-TLS handshake. While the clients also maintains connections with several other servers including a signaling server, these connections are not involved in the recovery sequence and are therefore omitted from the discussion.

Session establishment is performed using ICE, and our inspection of TURN message headers indicates that Teams does not comply with the standard defined in RFC8656 [16], but instead follows MS-TURN [2].

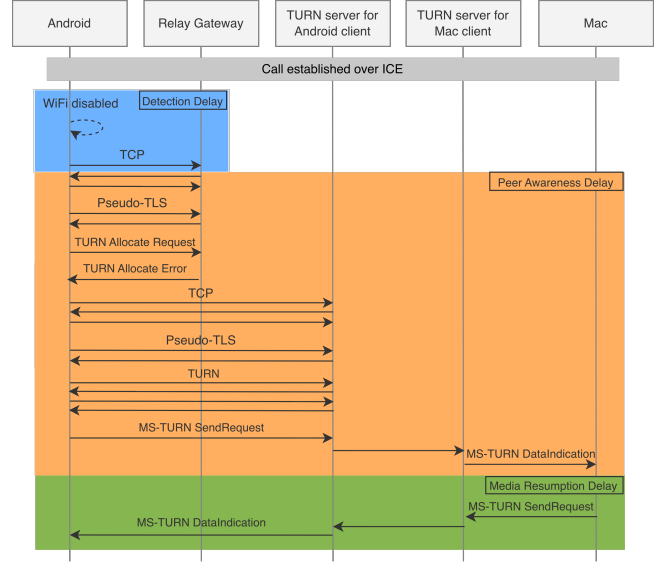


Figure 2: Microsoft Teams recovery sequence after an IP address change

From NetworkRequest entry from dumphys output, we observe that Microsoft Teams detects an IP address change from OS-level notification. After detecting an IP address change, the Android first performs a TCP handshake with the Relay Gateway and sends a TURN Allocate Request. We define the SYN packet toward the Relay Gateway as the end of the *detection delay*. The Relay Gateway responds with an Allocate Error Response containing an `ALTERNATE-SERVER` attribute, redirecting the client to another TURN server. The client then sends a TURN Allocate Request to the indicated server and establishes a new TCP connection to it. In all trials, the redirected TURN server matches the relay candidate assigned to the Android prior to the IP address change. Almost simultaneously with the TURN reconnection, the Android sends STUN Binding Requests to previously gathered candidates without changing the username, indicating that Teams employs ICE Renomination.

On the Mac side, once the Android reconnects to the TURN server, a TURN Data Indication message is received. Because responses to STUN Binding Requests on the P2P path have already ceased, the relayed path becomes the only valid path, and the Mac immediately switches media transmission to the relay path and sends a Send Request. We define the reception of this Data Indication at the Mac as the end of the *peer awareness delay*. After the Mac switches media transmission, a corresponding Data Indication carrying media arrives at the Android, which we define as the end of the *media resumption delay*.

The analysis shows that the *peer awareness delay* dominates the overall interruption time. This overhead results

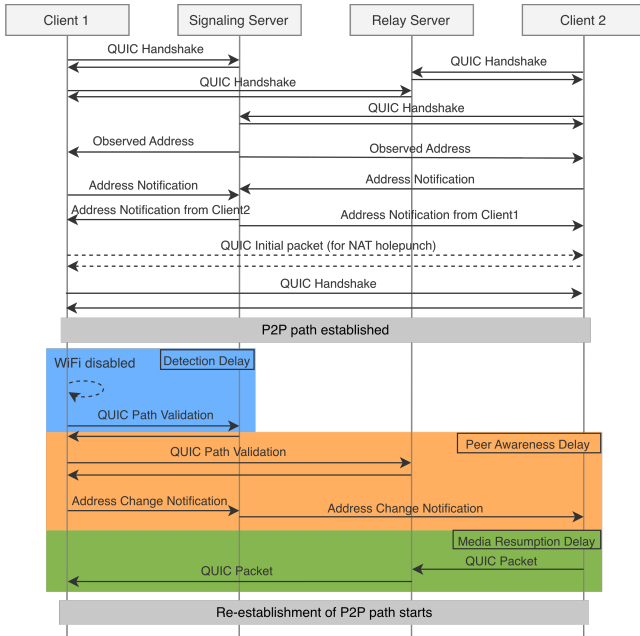


Figure 3: Sequence of QSwitch connection establishment and recovery procedure

from repeated TCP and TURN connection establishments toward the Relay Gateway and the redirected TURN server.

3.3 Key Insights

Zoom exhibits the longest interruption because the sender peer waits for 10 s to give up on the P2P path. Teams reduces interruption by employing ICE Renomination. However, Teams still suffers from multiple repetitive handshakes of TCP and TURN between servers. This suggests that reducing interruption requires avoiding repeated server connections and transport re-establishment during recovery.

4 QSwitch Design

QSwitch is a QUIC-based recovery approach under WiFi-to-cellular transitions for P2P conferencing systems. Because QUIC [10] manages connections using Connection IDs rather than the conventional 4-tuple, a client can resume an existing connection after an IP address change with a single round-trip path validation. Leveraging the insight from Section 3 that peer awareness delay dominates interruption time, QSwitch uses QUIC connection migration to immediately reestablish connectivity with the server upon an address change, notify the remote peer, and fall back to server-relayed media transmission. This design minimizes the *peer awareness delay*, the delay before the remote peer becomes aware of the address change.

4.1 P2P Session Establishment

The upper half of Figure 3 illustrates the P2P session establishment sequence of QSwitch. Each client first establishes QUIC connections to both the signaling server and the relay server. The signaling server returns the externally observed address of the client. Each client then sends an Address Notification, containing its local and global addresses, to the remote peer via the signaling server.

Upon receiving the peer’s addresses, each client sends QUIC Initial packets to both the local and global addresses of the peer. Connectivity attempts prioritize the local address over the global address. Once a valid path is established, media transmission proceeds over the P2P path.

4.2 Recovery Procedure

The lower half of Figure 3 illustrates the recovery sequence under an IP address change. When a client’s IP address changes, it detects the event immediately via a netlink notification. The client then resumes its QUIC connections with the signaling server and the relay server using a single round-trip path validation enabled by QUIC connection migration.

After reconnecting to the signaling server, the client sends an Address Change Notification, including its updated local address, to the remote peer via the signaling server. The signaling server appends the newly observed global address and forwards the notification to the remote peer. Upon receiving the notification, the remote peer immediately switches media transmission to the relay server. Media delivery is thus restored via the relay path.

After media transmission resumes, the peers re-establish the P2P path using the same procedure described in Section 4.1. Once a valid P2P path is restored, media transmission is switched back from the relay path.

5 Implementation

We implement QSwitch as a standalone prototype. The client and servers are written in Go and built on top of quic-go (v0.58.0). A QSwitch client consists of approximately 2,500 lines of code and runs on both Linux and macOS. It is composed of several components coordinated by an orchestrator module that manages connection state and peer address information. A monitoring component detects IP address changes using netlink notifications. Upon detecting an address change, it notifies the orchestrator, which triggers the recovery procedure described in Section 4.2.

The orchestrator manages three main components. Hole punch component is responsible for NAT hole punching to establish and re-establish the P2P path. Control plane component handles signaling messages exchanged with the signaling server, including Address Notification and Address Change Notification. Data plane component manages media

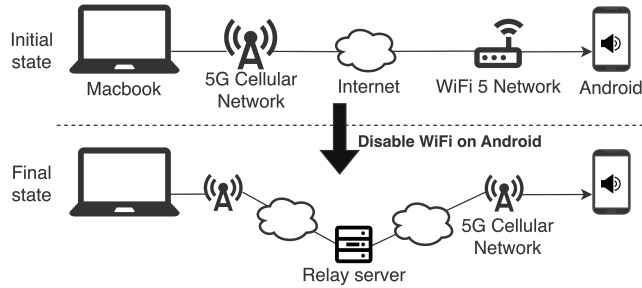


Figure 4: Experimental setup for WiFi-to-cellular transition experiments.

transmission over QUIC connections, including switching between the P2P path and the relay path. The prototype supports audio transmission from an MP3 file.

QSwitch uses two server components: a signaling server and a relay server. Both consist of fewer than 400 lines of code each. The signaling server maintains QUIC connections with clients and forwards control messages between peers. The relay server forwards media packets between peers when P2P path becomes invalid.

6 Evaluation

We evaluate the effectiveness of QSwitch in reducing interruption time under IP address changes. We first measure user-perceived interruption time by measuring silence duration from audio recordings. Second, we analyze its breakdown across phases to understand where improvements are achieved.

6.1 Baseline Implementation

We compare QSwitch with a WebRTC baseline built on Pion [15]. The baseline uses Pion’s ICE implementation with renomination, continual gathering, and UDP/IPv4 candidates enabled. Before handover, the baseline excludes cellular interfaces from candidate gathering so that media is sent over the original P2P path. When netlink reports WiFi address removal, it switches candidate gathering to the cellular interface, only gathers relay candidates through TURN, and forces nomination of a relay-to-relay candidate pair consisting of a cellular-backed local relay candidate and a remote relay candidate. This represents an aggressive WebRTC relay-fallback baseline that reacts to the same handover trigger as QSwitch, rather than waiting for the WebRTC stack’s default reconnection behavior. The netlink-based detection logic, the MP3 file containing the 440 Hz tone, and the receiver-side recording logic are shared with the QSwitch implementation.

6.2 Measurement Setup

Figure 4 illustrates our experimental setup. The experiments use a Nothing Phone 1 running Android 13 as the receiver

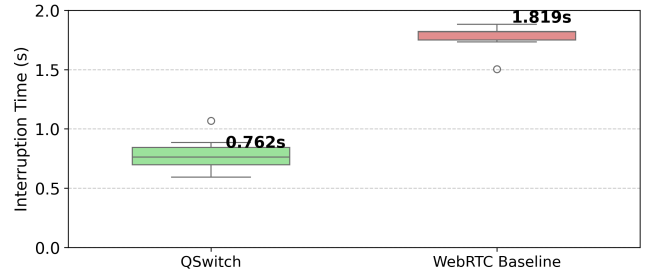


Figure 5: User-perceived interruption time.

and a MacBook Pro M2 (2022) as the sender. The peers start from different networks, both behind full-cone NATs. The Android switches from WiFi to a 5G cellular network, which we also observe to use a full-cone NAT. In each trial, the Mac continuously transmits the 440 Hz tone from the MP3 file to the Android, and we disable WiFi on the Android during the trial to force a WiFi-to-cellular transition. We record the received audio and collect packet traces and Android logs to measure interruption time and analyze the recovery phases. To avoid differences in client-server latency, the STUN/TURN and signaling servers used by the WebRTC baseline and the signaling and relay servers used by QSwitch run on Linux servers in the same location.

6.3 User-perceived Interruption Time

To quantify interruption time from the user’s perspective, we record the audio received at the Android device and measure periods of silence in each recording. Silence is detected automatically by treating samples below -35 dBFS as inaudible. The longest silent interval in a recording is defined as the interruption time for that trial.

Figure 5 shows the user-perceived interruption time over 10 runs. QSwitch achieves a median interruption time of 0.762 s, while the WebRTC baseline exhibits 1.819 s. QSwitch reduces the median interruption time by 58.1% compared to the WebRTC baseline. This result shows that QSwitch can reduce user-perceived interruption even compared with an aggressive WebRTC-based relay fallback using ICE renomination.

6.4 Breakdown of Interruption by Phase

We further decompose the interruption time into the three phases defined in Section 3. For QSwitch, the phase boundaries are defined as follows:

- **Detection delay** ends when the Android sends a QUIC path validation packet.
- **Peer awareness delay** ends when the Mac receives an Address Change Notification.
- **Media resumption delay** ends when the Android receives the first relayed media packet.

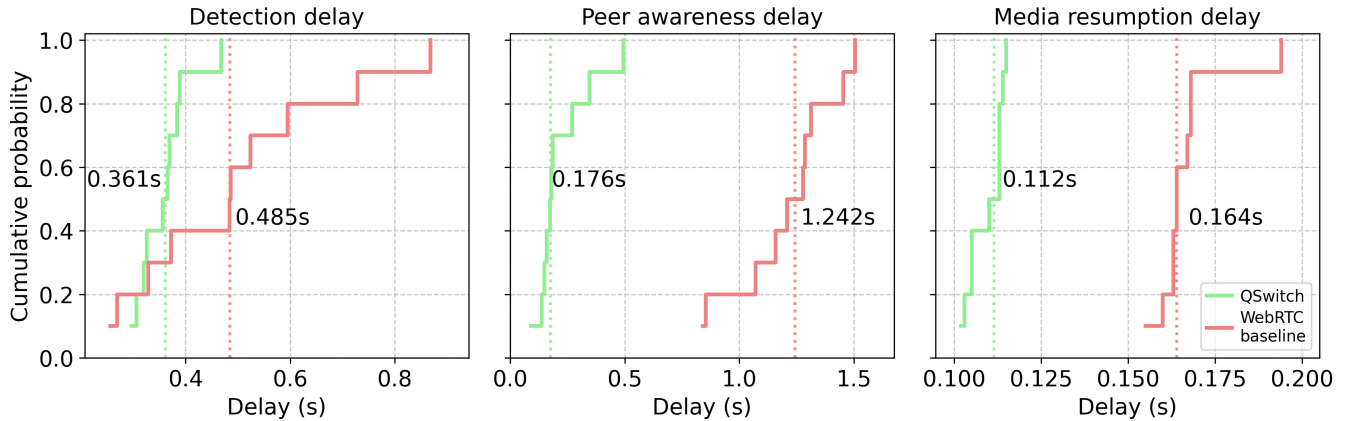


Figure 6: ECDF of interruption-time breakdown by recovery phase. Vertical dotted lines and numbers indicate the median of each distribution.

For the WebRTC baseline, the phase boundaries are defined as follows:

- **Detection delay** ends at the first SYN packet sent to reconnect to the signaling server.
- **Peer awareness delay** ends when the Mac receives a TURN Data Indication.
- **Media resumption delay** ends when the Android receives the first TURN ChannelData message.

Figure 6 shows that QSwitch achieves lower delays across all three phases, with the largest difference observed in peer awareness delay. QSwitch reduces the median peer awareness delay from 1.242 s to 0.176 s, while the differences in detection delay and media resumption delay are smaller in absolute terms.

In the WebRTC baseline, the Android client first reconnects to the signaling server over TCP after the address change. It then re-authenticates with the TURN server from the new address by sending an Allocate Request, exchanges CreatePermission messages, and sends a Send Indication through TURN. These sequential control-plane procedures increase the peer awareness delay. In contrast, QSwitch keeps the server-facing QUIC connections alive across the address change and notifies the peer after QUIC path validation, avoiding repeated transport and TURN setup during recovery.

7 Limitation

Emulated WiFi-to-Cellular Transitions. In our experiments, network transition is emulated by explicitly disabling the WiFi interface, which causes an immediate switch to the cellular network. In real deployments, however, connectivity typically degrades gradually due to worsening signal quality or increasing packet loss, and the operating system switches the primary interface only after sustained degradation rather

than a discrete failure. Consequently, interruption behavior in practice may differ from that observed in our controlled setup. Future work should incorporate mechanisms for continuously monitoring network conditions and proactively initiating path migration.

Media Transport over QUIC. Another limitation of QSwitch is the use of QUIC for media transport. Modern real-time conferencing systems are largely built on WebRTC [6], whose major implementations provide mature and highly optimized media stacks including RTP-based media transport, congestion control, jitter buffering, packet pacing, and adaptive bitrate mechanisms integrated with ICE-based connectivity establishment. These components have been refined over 10 years of deployment. In contrast, QUIC primarily provides a general-purpose transport protocol and does not natively include media-layer mechanisms. While Media over QUIC [14] is proposed as a payload-agnostic media transport protocol over QUIC, deploying QUIC-based media transport in production conferencing systems would still require substantial engineering effort to match the performance and robustness of existing WebRTC-based solutions.

8 Conclusion

We analyze recovery procedures in existing P2P conferencing systems under IP address changes and identify that interruption time is dominated by the delay before the remote peer becomes aware of the address change. Based on this insight, we present QSwitch, a QUIC-based mechanism that enables rapid fallback without repeated transport handshakes. Evaluations on real devices demonstrate that QSwitch reduces user-perceived interruption by 58.1% compared to a controlled WebRTC baseline using ICE renomination.

References

- [1] 2024. Enabling Peer-to-peer connection for 2 people in a meeting. https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0067877 [Online; accessed 2026-01-31].
- [2] 2025. MS-TURN: Traversal Using Relay NAT (TURN) Extensions. https://learn.microsoft.com/en-us/openspecs/office_protocols/ms-turn/9e434b27-eb13-4249-b031-2d15c3835c8b [Online; accessed 2026-02-28].
- [3] 2026. Start a call from a chat in Microsoft Teams - Microsoft Support. <https://support.microsoft.com/en-us/office/start-a-call-from-a-chat-in-microsoft-teams-f5138c9d-df4c-43d8-9cf6-53400c1a7798> [Online; accessed 2026-01-31].
- [4] Rahul Bothra, Rohan Gandhi, Ranjita Bhagwan, Venkata N. Padmanabhan, Rui Liang, Steve Carlson, Vinayaka Kamath, Sreangsu Acharyya, Ken Sueda, Somesh Chaturmohta, and Harsha Sharma. 2023. Switchboard: Efficient Resource Management for Conferencing Services. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) (ACM SIGCOMM '23). Association for Computing Machinery, New York, NY, USA, 1000–1011. doi:10.1145/3603269.3604879
- [5] Andreas Bovens. 2026. P2P vs SFU Video Calls: Which is Best? | Whereby. <https://whereby.com/blog/p2p-vs-sfu-video-calls-which-is-best/> [Online; accessed 2026-01-31].
- [6] Florent Castelli, Henrik Boström, Cullen Jennings, and Jan-Ivar Bruaroey. 2024. *WebRTC: Real-Time Communication in Browsers*. W3C Recommendation. W3C. <https://www.w3.org/TR/2024/REC-webrtc-20241008/>.
- [7] Wesley Eddy. 2022. Transmission Control Protocol (TCP). RFC 9293. doi:10.17487/RFC9293
- [8] Jia He, Mostafa Ammar, Ellen Zegura, Emir Halepovic, and Theo Karagioules. 2024. QoE Metrics for Interactivity in Video Conferencing Applications: Definition and Evaluation Methodology. In *Proceedings of the 15th ACM Multimedia Systems Conference* (Bari, Italy) (MM-Sys '24). Association for Computing Machinery, New York, NY, USA, 178–189. doi:10.1145/3625468.3647622
- [9] Emil Ivov. 2017. P2P4121 - Jitsi. <https://jitsi.org/blog/p2p4121/> [Online; accessed 2026-02-01].
- [10] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000. doi:10.17487/RFC9000
- [11] Mehdi Karamollahi, Carey Williamson, and Martin Arlitt. 2023. Packet-Level Analysis of Zoom Performance Anomalies. In *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering* (Coimbra, Portugal) (ICPE '23). Association for Computing Machinery, New York, NY, USA, 221–232. doi:10.1145/3578244.3583725
- [12] Ari Keränen, Christer Holmberg, and Jonathan Rosenberg. 2018. Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal. RFC 8445. doi:10.17487/RFC8445
- [13] Oliver Michel, Satadal Sengupta, Hyojoon Kim, Ravi Netravali, and Jennifer Rexford. 2022. Enabling passive measurement of zoom performance in production networks. In *Proceedings of the 22nd ACM Internet Measurement Conference* (Nice, France) (IMC '22). Association for Computing Machinery, New York, NY, USA, 244–260. doi:10.1145/3517745.3561414
- [14] Suhas Nandakumar, Victor Vasiliev, Ian Swett, and Alan Frindell. 2026. *Media over QUIC Transport*. Internet-Draft draft-ietf-moq-transport-17. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-ietf-moq-transport/17/> Work in Progress.
- [15] Pion. [n. d.]. Pion: The Modern Stack for Web Real-Time Communication. <https://pion.ly/>. Accessed: 2026-05-02.
- [16] Tirumaleswar Reddy, K. Alan Johnston, Philip Matthews, and Jonathan Rosenberg. 2020. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN). RFC 8656. doi:10.17487/RFC8656
- [17] Peter Thatcher, Honghai Zhang, and Taylor Brandstetter. 2016. *ICE Renomination: Dynamically selecting ICE candidate pairs*. Internet-Draft draft-thatcher-ice-renomination-01. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-thatcher-ice-renomination/01/> Work in Progress.
- [18] Matteo Varvello, Hyunseok Chang, and Yasir Zaki. 2022. Performance characterization of videoconferencing in the wild. In *Proceedings of the 22nd ACM Internet Measurement Conference* (Nice, France) (IMC '22). Association for Computing Machinery, New York, NY, USA, 261–273. doi:10.1145/3517745.3561442
- [19] Kabir Sikand Will Moore, Kevin Purdy. 2025. How Tailscale is improving NAT traversal (part 1). <https://tailscale.com/blog/nat-traversal-improvements-pt-1> [Online; accessed 2026-03-03].